解説資料:AI 海難救助ドローン

レポート編(説明資料 1〜6 章の詳しい解説)

Physical AI 応用講座1 最終課題(海難救助版)の付録 ／ 2026 年 10 月 ／ コード: GitHub ba2vwbus13/pai-ai-sea-rescue-drone

この資料について

提出した説明資料(4 ページ)は、要点だけを短く書いています。この解説資料は、その各章を図と表を使って詳しく説明したものです。説明資料と同じ章立てにしているので、説明資料を読んでわかりにくかった章を、こちらで開いてください。

実装(プログラムの中身)の解説は、この資料のあとに「実装編」として追加する予定です。

目次

1. 課題の概要と想定(状況の地図、ドローンが知っていること)
2. システム構成(4 つのプログラムの役割、状態の流れ、目的地と仮の目標点)
3. 工夫した点(漂流予測と拡大方形捜索、動く人の追跡と投下、陸上版から引き継いだ 4 つの工夫、公平な評価)
4. 結果(表の読み方、一人ずつの結果、誤差が生まれる理由)
5. 考察と今後の課題(単純にしたこと、新たに見つかった課題)
6. 使用した技術

用語の約束

用語	この資料での意味
mission(頭)	飛行の手順を決めるプログラム(`mission_node.py`)。2 章で説明する 4 つのプログラムの 1 つ
mission(頭)から出力した JSON ファイル	mission が着陸したときに書き出す記録(`results/mission_*.json` 。GitHub では `sea_run1〜3.json`)
目的地	状態ごとに決まる「最終的に止まる場所」
仮の目標点	目的地へ向かって 0.1 秒ごとに少しずつ進む点。これを PX4 に送る(コードでは `carrot`)
確信度	AI が「人である」とどれだけ思っているかを 0〜1 で表した数字
T 番号(T1, T2 …)	mission が、見つけた候補に付けた通し番号(答案の側)
P 番号(P1〜P5)	ワールドを作るときに付けた、要救助者の本当の名前(答えの側)
位置と速度の表し方	離陸地点(砂浜のヘリパッド)を原点に、「北に何 m・東に何 m」の 2 つの数で表す(PX4 と同じ方式)。北と東がプラスなので、**北 −152 m は南へ 152 m、東 −47 m は西へ 47 m**。速度も同じで、北向き −0.075 m/s は南へ 0.075 m/s(3.2 節の図 3-3)
UNIX 時刻	1970 年 1 月 1 日から何秒たったかで表した時刻(例: 1791880498.60)。2 つのファイルの時刻を合わせるのに使う

1 章 課題の概要と想定

1.1 何をしたか

説明資料の「ドローンが流された位置を予測して自律的に捜索し、AI で漂流者を見つけて確かめ、救命浮環を投下する」という 1 文には、ドローンの仕事が 4 つ入っています。

仕事	平たく言うと	動画のどこに出るか
位置を予測する	「今ごろ、どこまで流されたか」を計算する	冒頭のログ「漂流予測: 通報地点から 94 m 流されたと推定」
自律的に捜索する	人が操縦せず、決まった飛び方で海を探す	地図上の四角い渦巻きの経路
見つけて確かめる	AI が人らしいものを見つけ、近づいてもう一度 AI で判定する	カメラ映像の赤い枠、「AI で再確認」の表示
救命浮環を投下する	人の真上から浮き輪を落とす	オレンジの輪が海に浮かぶ場面

陸上版(災害時の要救助者捜索)との違いは、海特有の 3 つの課題です。これらが 3 章の工夫につながります。

- **対象が動く**:陸の人は止まっているが、海の人は流され続ける → 動く人の追跡(3.2)
- **捜索の起点を推定する必要がある**:通報された地点に、もう人はいない → 漂流予測(3.1)
- **物を届ける必要がある**:見つけて終わりではない → 救命浮環の投下(3.2)

1.2 想定の地図

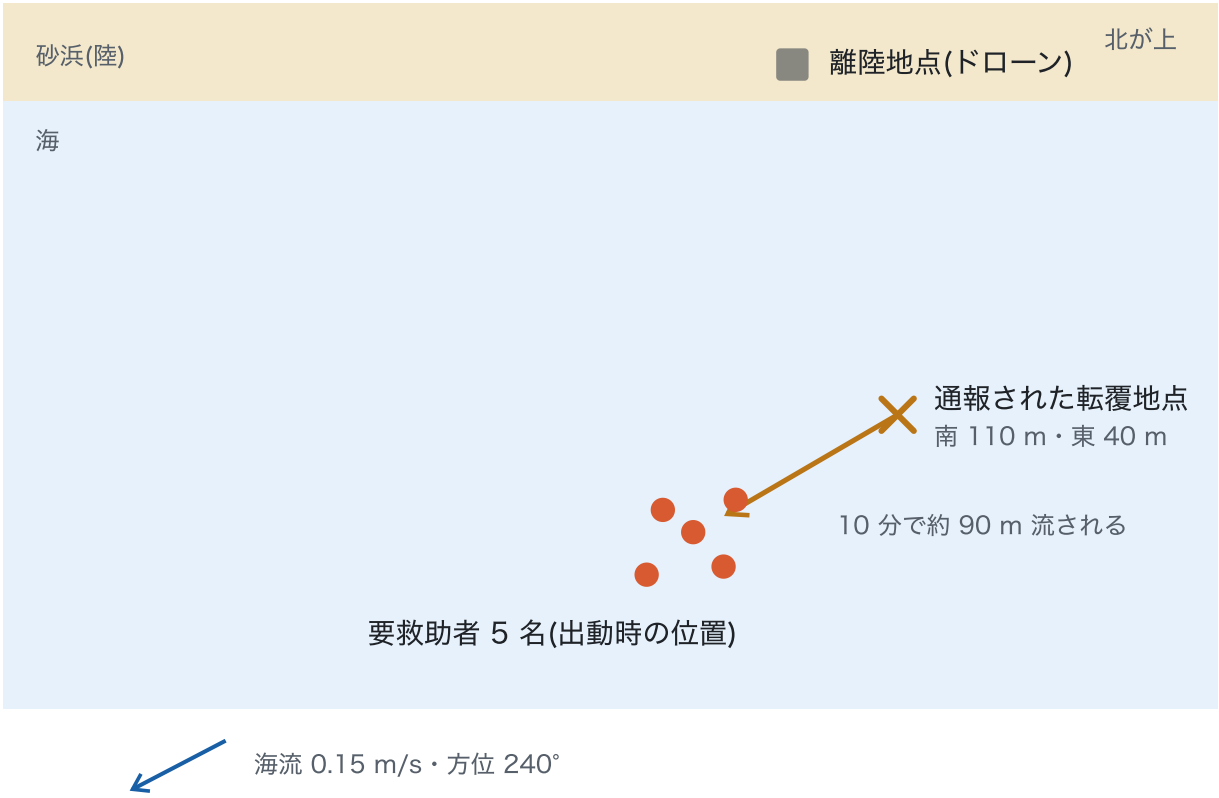


図 1-1 想定の位置関係(模式図。縮尺はおおよそ)

表 1-1 想定の数字の読み方

数字	意味
海流 0.15 m/s	1 秒に 15 cm、1 分に 9 m 流される。出動までの 10 分で約 90 m(図の茶色の矢印)
方位 240°	方位は北を 0° として時計回りに数える(東 90°、南 180°、西 270°)。240° は南と西の間で、やや西寄り
乗員 5 名	浮いている人 2 名、立ち泳ぎの人 3 名。立ち泳ぎの人は胸まで水中なので、上空からは体の一部しか見えない
通報から 10 分後に出動	この間も人は流され続ける。海流が 2 倍(0.3 m/s)なら約 180 m 離れる

1.3 ドローンが知っていること、知らないこと

ドローンが知っていること	ドローンが知らないこと
通報された転覆地点(南 110 m・東 40 m) 乗員が 5 名いること 海流の予報(0.15 m/s・240°)	要救助者が今どこにいるか それぞれがどれだけ散らばっているか

> シミュレーションの中では、プログラムは要救助者の本当の位置を知ろうと思えば知ることができます。それをドローンの制御に使うと、答えを見ながら試験を解くのと同じになります。そこで、本当の位置は**採点のときだけ**使い、ドローンは実際の救助と同じ情報だけで動くようにしました(3.4 節)。

1.4 要救助者の散らばり方(簡略化)

結果の地図(4 章)では、5 名の漂流の跡(青い線)が平行で、間隔も変わりません。これは次のように簡略化したためです。

> 実際の海では、人は風の受け方(浮いている人と立ち泳ぎの人では違う。海難救助では「リーウェイ」と呼ぶ)、泳ぐ・船につかまるといった動きの違いによって、転覆地点から**時間とともに**散らばっていきます。今回は簡単のために、**転覆した瞬間に散らばりを持たせ**、その後は全員を同じ海流で**等速直線**に流しています。

表 1-2 転覆した瞬間の各人の位置(転覆地点からのずれ。worlds/make_world.py の PERSONS)

人	姿勢	北	東	備考
P1	浮いている	+4 m	−6 m	
P2	立ち泳ぎ	−10 m	+8 m	
P3	立ち泳ぎ	+12 m	+14 m	
P4	立ち泳ぎ	−45 m	−30 m	拡大方形捜索が必要になるよう、あえて遠くに置いた(近いと、捜索基点へ向かう途中で全員見つかってしまった)
P5	浮いている	+40 m	−55 m	

2 章 システム構成

2.1 中心となる 4 つのプログラム

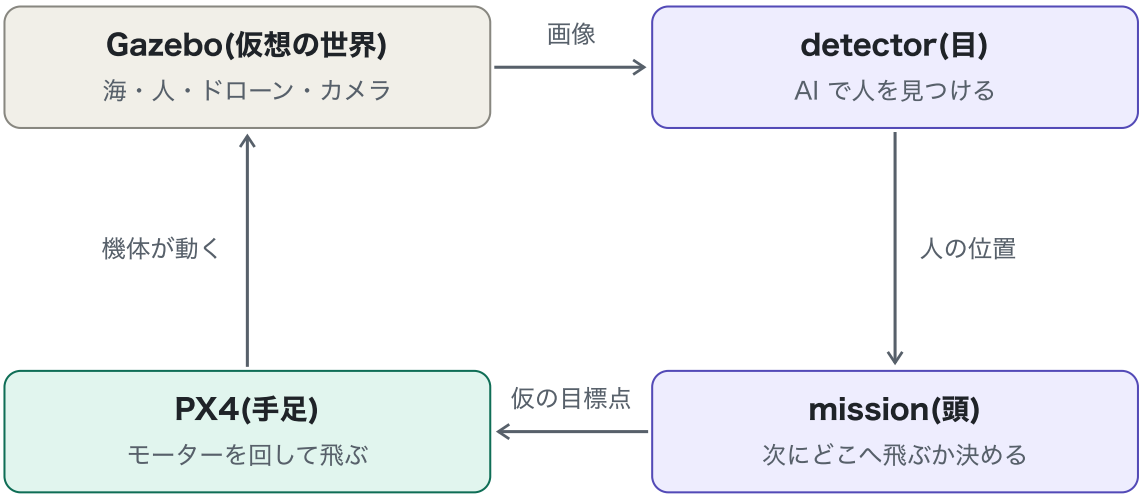


図 2-1 4 つのプログラムの一周。この流れがぐるぐる回り続ける(機体が動く → 景色が変わる → 人を見つける → 近づく目標を決める → 飛ぶ …)

表 2-1 4 つのプログラムの役割

プログラム	たとえ	受け取るもの	渡すもの	頻度
Gazebo	仮想の世界	ドローンのモーターの回転	カメラ画像、機体の位置・傾き	常に計算
detector	目	カメラ画像、機体の位置・傾き(PX4 から)	「人がいる、確信度 0.8、位置は北に −152 m・東に −47 m」	1 秒に 1 回
mission	頭	人の位置、機体の位置	仮の目標点(位置)+ そこへ向かう速度 + 機首の向き	1 秒に 10 回
PX4	手足	仮の目標点・速度・機首の向き	モーターの回転(結果として機体が動く)	常に制御

PX4 は「どこへ行くか」を考えません。「言われた地点へ、安定して飛ぶ」ことだけを担当します。どこへ行くかを決めるのは mission です。今回作ったのは主に detector と mission で、PX4 と Gazebo は既製のソフトです。

detector は、人の位置を計算するために PX4 から機体の位置と傾きも受け取っています。画像の中のどこに人が写っているかだけでは、海の上の位置は決まらないからです(3.3 節の工夫 4)。

2.2 中心の流れの外にある 3 つのプログラム

プログラム	役割	補足
payload	救命浮環の投下装置	mission から「投下せよ」を受けて、Gazebo の中に浮環を出現させ、落下させ、浮かべる
dashboard	表示画面	動画の左側と右下の画面。情報を集めて表示するだけで、飛行には影響しない
truth_logger	採点用の記録係	Gazebo から要救助者と浮環の本当の位置を記録する。mission には渡さないので、止めてもドローンの動きは変わらない(採点ができなくなるだけ)

2.3 状態の流れ(3 回目の飛行=デモ動画)

mission は「今、何をしているか」の段階(状態)を切り替えながら動きます。3 回目の飛行の記録で、実際の流れを追います。時刻は出動(アーム)からの秒数です。

表 2-2 3 回目の飛行の流れ(mission(頭)から出力した JSON ファイルの events より)

時刻	状態	何が起きたか
0 秒	離陸	砂浜から上昇
9.7 秒	漂流予測 → 捜索基点へ移動	「94 m 流された」と計算し、その地点へ向かう
21.7 秒	発見 → 候補へ移動	移動中に AI が T1 を見つける(確信度 0.84)
21.7〜30.9 秒	降下して確認・AI で再確認	高度 10 m に下りて、AI でもう一度判定する
30.9 秒	確定	T1 を要救助者と確定(確信度 0.83)
30.9〜35.7 秒	救命浮環を投下	高度 6 m で真上に止まり、投下する
36.6〜51.4 秒	候補 T2・T3 を発見 → T3 を確認	T3 は近距離で見えず、誤検出として棄却
54.2〜66.3 秒	T2 を確認・確定・投下	
72.7〜194.4 秒	T4・T5・T6 を同様に	T5・T6 は拡大方形捜索を進める中で見つかる
198.5 秒	帰還	乗員 5 名全員を確定したので、捜索を打ち切る
240.0 秒	着陸	JSON ファイルを書き出して終了

2.4 目的地と仮の目標点

mission は 0.1 秒ごとに、次の 3 段階で PX4 への指示を作っています。

1. 今の状態に応じて**目的地**を選ぶ(表 2-3)
2. 人に関わる状態では、人の**予測位置**から目的地を計算する(図 2-2)
3. 目的地をいきなり送らず、そこへ向かって少しずつ進む**仮の目標点**を作って PX4 に送る(表 2-4)

表 2-3 状態ごとの目的地(目的地から 1.5 m 以内に入ったら「到着」として次の状態へ)

状態	目的地(水平の位置)	高度
離陸	離陸地点の真上	18 m
捜索基点へ移動	漂流を補正した捜索基点	18 m
拡大方形捜索	捜索経路の次の曲がり角	18 m
A 候補へ移動	人の予測位置の手前 10.4 m	18 m
B 降下して確認・AI で再確認	人の予測位置の手前 5.8 m	10 m
C 救命浮環を投下	人の予測位置の真上	6 m
上昇	今いる場所の真上	18 m
捜索へ復帰	捜索を中断した地点	18 m
帰還	離陸地点	18 m

人に近づくとき、目的地を人の真上ではなく**手前**にしているのは、カメラが真下ではなく前方斜め下 30° を向いているからです。真上に行くと、人がカメラの画面から外れます。人が画面の真ん中に写る位置、つまり「高度 × tan 30°」だけ手前を目的地にします(高度 18 m なら 10.4 m、10 m なら 5.8 m)。

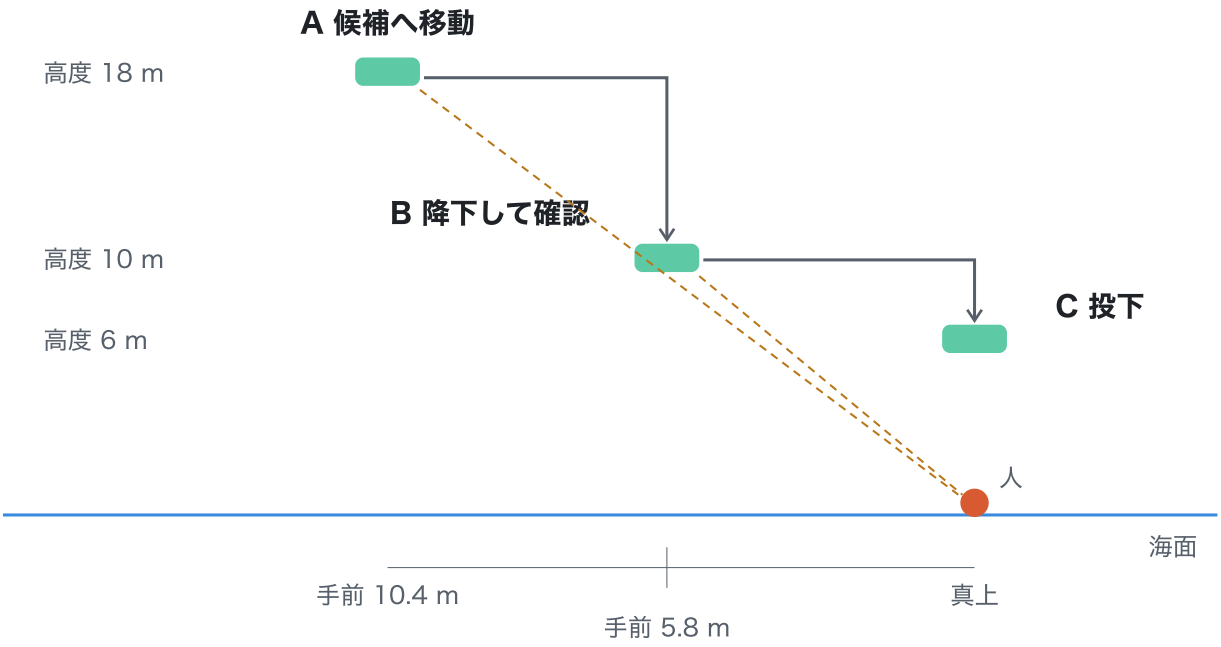


図 2-2 横から見た 3 つの目的地。オレンジの点線はカメラの中心で、A と B では人が画面の真ん中に写る。灰色の矢印は実際の動き方で、水平に先に進んでからほぼ真下に降りる(くの字)

A でドローンは止まり、すぐ B へ切り替えます。B では止まって最低 3 秒、AI で観察します(再確認)。確定したら C へ移り、1.5 秒静止してから浮環を落とします。止まっている間も、人が海流で流れる分(1 秒に 0.15 m)だけ目的地は動くので、ドローンも人と一緒にゆっくり横にずれます。

仮の目標点

目的地が 100 m 先にあるとき、その地点をそのまま PX4 に送ると、PX4 は全力で加速します。速度を自分で管理するために、mission は目的地へ向かって一定の速さで進む**仮の目標点**を作り、0.1 秒ごとに送ります。水平は

秒速 7 m(0.1 秒で 0.7 m)、上下は秒速 3 m(0.1 秒で 0.3 m)と、別々の速さで動かします。

表 2-4 A → B の間の仮の目標点(目的地はずっと B)

経過時間	前へ進んだ距離	高度	説明
0 秒	0 m	18.0 m	A を出発
0.1 秒	0.7 m	17.7 m	
0.7 秒	4.6 m	15.9 m	水平は先に B の真上に着く
2.0 秒	4.6 m	12.0 m	ここからは真下に降りるだけ
2.7 秒	4.6 m	10.0 m	B に到着(この間に約 27 回、毎回違う点を送る)

表 2-5 目的地と仮の目標点の違い

	目的地	仮の目標点
決め方	状態ごとに決まる(表 2-3)	目的地へ向かって 0.1 秒ごとに少しずつ進める
PX4 に送るか	送らない(mission の中だけで使う)	送る(位置 + 速度 + 機首の向き)
A → B の間	ずっと B	毎回変わる

PX4 は、仮の目標点を 1 秒に 2 回以上受け取り続けないと「外部からの指示が途絶えた」と判断して安全動作に切り替わります。そのため、目的地に着いて止まっているときも、同じ点を 1 秒に 10 回送り続けています。

3 章 工夫した点

3.1 漂流予測と拡大方形捜索:どこから探すか

漂流予測

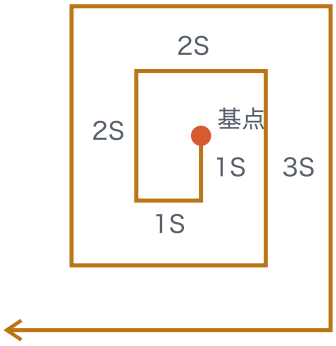
ドローンは高度 18 m に着いた時点で、「流された距離 = 海流の速さ × 通報からの経過時間」を計算し、その分だけ海流の向きにずらした地点を**捜索基点**にします。

表 3-1 経過時間の内訳(mission_node.py の _plan_search)

項目	値	中身
通報から出動まで	600 秒	パラメータ incident_elapsed_s
mission の起動から今まで	約 15 秒	アームを待つ時間と、高度 18 m まで上がる時間
捜索基点までの移動時間の見込み	11.7 秒	通報地点までの距離 117 m ÷ 移動速度 10 m/s
合計	**約 627 秒**	0.15 m/s × 627 秒 = **94 m**(動画のログの値)

効果:捜索を始めた時点で、要救助者 5 名の重心までの距離は、通報地点からだと約 107 m、捜索基点からだと約 15 m でした。15 m が 0 にならないのは、5 名が最初から散らばっているからです(1.4 節)。

拡大方形捜索



脚の長さ: 1S, 1S, 2S, 2S, 3S, 3S…

1 本進むごとに右へ 90° 曲がる

S = 32 m(カメラの撮影幅 43 m × 0.75)

最初の脚は海流の下流の向き(240°)

脚は 10 本まで

図 3-1 拡大方形捜索。見やすいよう上下左右に描いたが、実際は最初の脚が下流(240°)を向くよう全体を回転させている(4 章の地図の斜めの点線の四角)

- **近い所から探す**:漂流予測が正しければ、人は基点の近くにいる可能性が一番高い
- **隙間を作らない**:脚の間隔 S を撮影幅の 75% にして、隣の脚と 25% 重ねる
- **最初は下流へ**:人は下流へ流れ続けるので、まずそちらを見る

国際的な捜索救助のマニュアル(IMO・ICAO の IAMSAR マニュアル)にある標準的な方法の 1 つで、事故の場所がある程度わかっているときに使います。

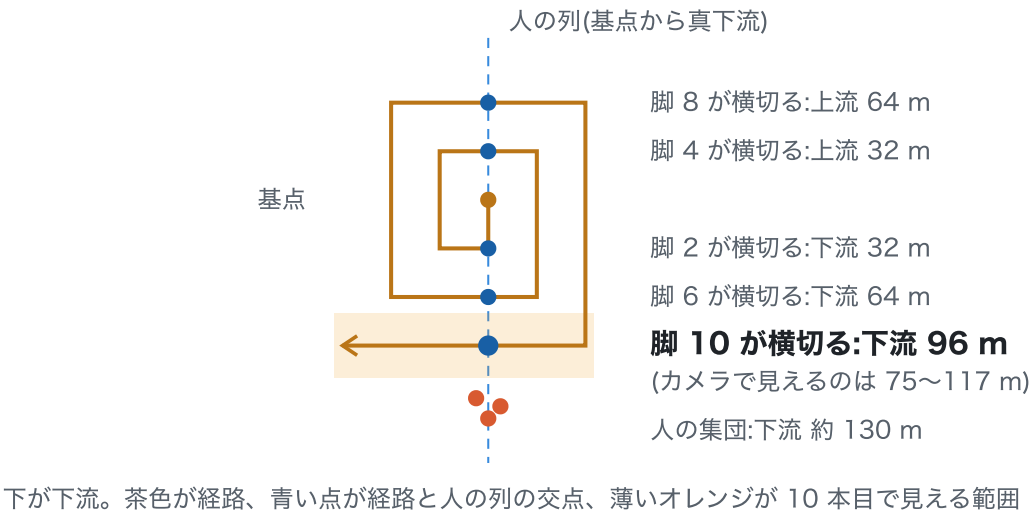
予報が外れるとどうなるか(限界)

捜索経路は、捜索を始める時点で地図の上に**固定**されます。一方、人は流れ続けます。例として、予報 0.15 m/s に対して実際の海流が 0.3 m/s だった場合を計算します。

表 3-2 予報が実際の半分だった場合

項目	値
実際に流された距離(出動時)	0.3 × 627 秒 ≈ 190 m(ドローンの見積もりは 94 m)→ 基点は集団より約 94 m 上流
人の列を一番遠くで横切る脚	10 本目。下流方向に基点から +96 m(3S)の線を横切る。捜索開始から約 2 分後
そのときカメラが見る範囲(下流方向)	96 ± 21 m = 75〜117 m
そのときの人の集団	94 + 0.3 × 120 秒 ≈ 130 m 下流
結論	130 m は 117 m の外 → 集団の中心は見つからないまま捜索が終わる可能性が高い

「人の列」とは、海流の向きは予報どおりで速さだけが違う場合に、人が並ぶ**基点から真下流の線**のことです。拡大方形捜索の脚のうち、奇数番目(1, 3, 5, 7, 9)は列と平行に進み、偶数番目(2, 4, 6, 8, 10)が列を横切ります。ドローンが列の真上を通るのは、主に偶数の脚が横切る瞬間です(図 3-2)。



下が下流。茶色が経路、青い点が経路と人の列の交点、薄いオレンジが 10 本目で見える範囲

図 3-2 拡大方形捜索の脚が人の列を横切る位置。9 本目は下流へ進むが、列から横に 64 m ずれた所を通るので(カメラが横に見えるのは 21 m まで)、列は見えない

脚の数はパラメータ `search_legs: 10` で 10 本と決めているので、列を一番遠くで横切るのは 10 本目です。では、脚を増やせば見つかるでしょうか。

表 3-3 脚を増やした場合(人の集団 = 94 m + 0.3 m/s × 経過時間、カメラの範囲 = 横切る位置 ± 21 m)

列を下流で横切る脚	横切る位置	時刻(捜索開始から)	カメラで見える範囲	人の集団	差
2 本目	32 m	約 6 秒	11〜53 m	約 96 m	43 m 届かない
6 本目	64 m	約 50 秒	43〜85 m	約 109 m	24 m 届かない
10 本目	96 m	約 123 秒	75〜117 m	約 131 m	14 m 届かない
14 本目	128 m	約 238 秒	107〜149 m	約 165 m	16 m 届かない
18 本目	160 m	約 401 秒	139〜181 m	約 214 m	33 m 届かない

横切る位置は脚 4 本ごとに 32 m しか進みませんが、脚はだんだん長くなるので、その 4 本を飛ぶ間に人は 32 m より多く流されます(10→14 本目で約 35 m、14→18 本目で約 49 m)。差は 10 本目あたりで一番近づいたあと、また広がります。**脚を増やしても集団の中心には追いつけません。**経路が地図の上に固定され、人は流れ続けるからです。経路そのものを海流で流せば、人は経路に対して止まって見えるので、脚を増やせばいつか必ず通過します。(集団の上流寄りにいる人は、範囲の端に入って見つかる可能性はあります。)

対策としては、捜索経路そのものを海流で流す(人と一緒に動かす)、予報の誤差を見込んで捜索範囲を広げる、見つけた人の動きから実際の海流を推定し直す、などがあります(5 章)。

3.2 動き続ける人の追跡と、救命浮環の投下

位置を「場所と時刻の組」で持つ

mission は、見つけた人の位置を「いつ、どこにいたか」の組で覚えています。使うときは、そのときの時刻に合わせて流れた分を足します。

予測位置 = 最後に推定した位置 + 海流の速度 ×（最後に推定してからの経過時間）

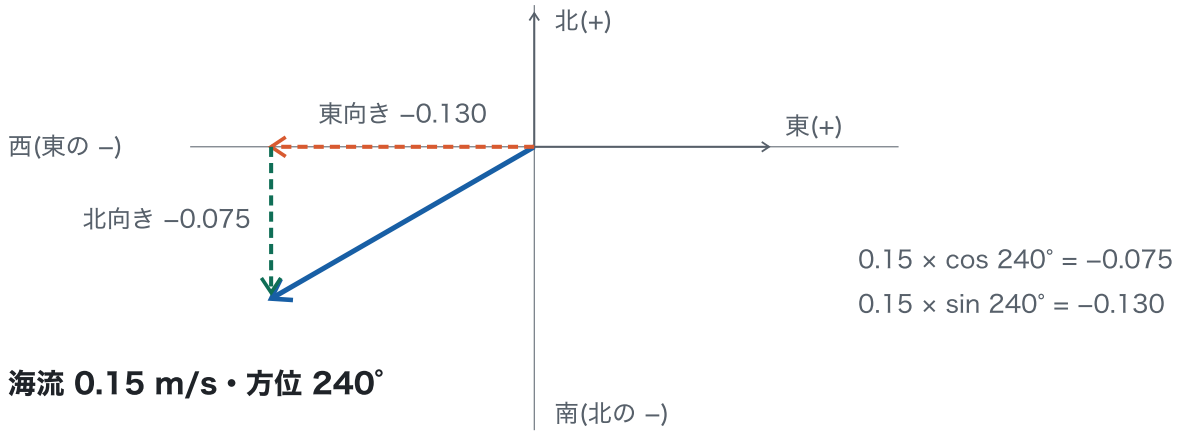


図 3-3 海流の速度を北向きと東向きの成分に分ける。どちらもマイナスなので、南西寄りの西へ流れる

例:AI が人を北 −152.0 m・東 −47.0 m(南へ 152.0 m・西へ 47.0 m)と推定した 8 秒後の予測位置は、

- 北:−152.0 + (−0.075 × 8)= −152.6 m(南へ 152.6 m。0.6 m 南へ流れた)
- 東:−47.0 + (−0.130 × 8)= −48.0 m(西へ 48.0 m。1.0 m 西へ流れた)

表 3-4 予測位置の 3 つの使い道

場面	使い方	使わないと起きること
照合	新しく見つけた人が、すでに知っている人と同じかを判断する(予測位置から 8 m 以内なら同じ人)	約 53 秒で 8 m 流れるので、同じ人を「新しい人」と勘違いし、何度も確認しに行く。浮環も同じ人に何個も落とす
接近	人の手前 10.4 m(5.8 m)の目的地を計算する	古い場所へ近づき、カメラに人が写らず、本物の人を「誤検出」として棄却してしまう
投下	人の真上の目的地を計算する	人から離れた場所に浮環が落ちる(1 分後なら 9 m)

確定位置は「時刻をそろえてから」平均する

表 3-5 観測を今の時刻にそろえて平均する例(説明のための数値)

観測した時刻	観測した位置(北, 東)	今の時刻にそろえた位置
0 秒前	(−152.6, −48.0)	(−152.6, −48.0)
1 秒前	(−152.5, −47.8)	(−152.6, −47.9)
2 秒前	(−152.4, −47.8)	(−152.6, −48.1)
平均		(−152.6, −48.0)

そろえずに平均すると、流れる前の古い位置が混ざり、平均が上流側にずれます。

救命浮環の投下

1. 予測位置の真上、高度 6 m で 1.5 秒静止する
2. payload が、機体の少し下(高度約 5.6 m)に浮環を出現させる
3. 自由落下させる(高さ = 5.6 − ½ × 9.8 × t²)。約 1.1 秒で海面に着く
4. 着水後は浮かべたまま、人と同じ海流で流す

> Gazebo には浮力の計算がありません。浮環は「重力を切り、動きをプログラムが式で与える」形で表現しています。落下と漂流は物理計算の結果ではありません。

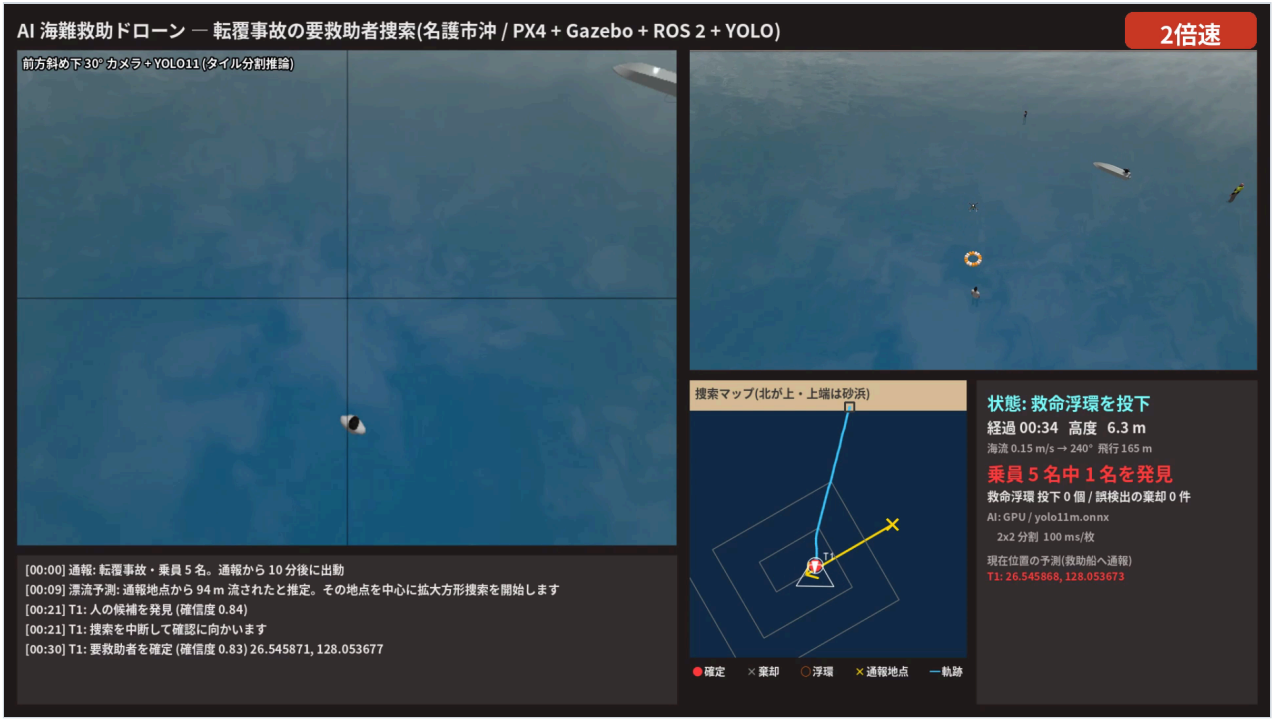


図 3-4 投下の場面。右上の 3D 画面で、浮環(オレンジ)が要救助者の横に浮いている

3.3 陸上版から引き継いだ 4 つの工夫

どれも「困りごと → 工夫 → 効果 → 代わりに払うもの」の形で整理します。

工夫 1:タイル分割推論

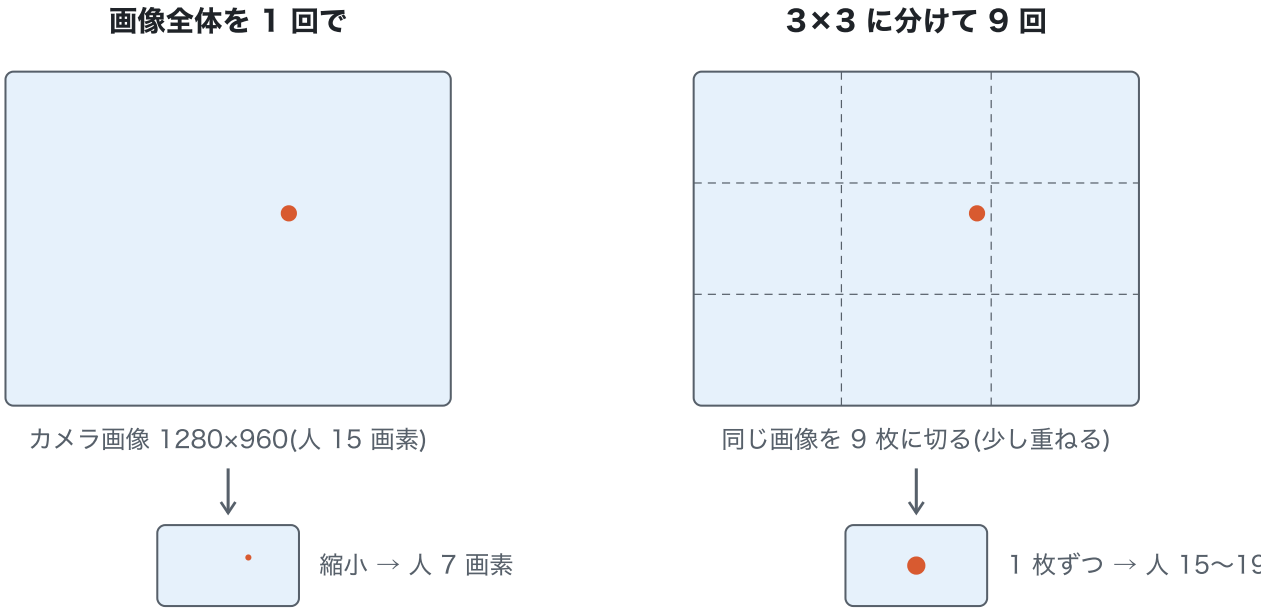


図 3-5 画像全体を縮小する方式と、タイル分割の比較

困りごと	AI(YOLO)は 640×640 画素の画像しか受け付けない。1280×960 のカメラ画像は半分に縮小され、高度 18 m で 15 画素ほどの人が約 7 画素になり、人だとわからなくなる
工夫	画像を 3×3 の 9 枚(1 枚 約 490×370 画素、隣と 2 割重ねる)に切り、1 枚ずつ AI に入れる。人は 15〜19 画素のまま残る。高度 12 m 未満では人が大きく写るので 2×2 に減らす
効果	陸上版の実験(高度 10〜20 m):全体を 1 回では 0 人、3×3 では 3 人全員を検出
代わりに払うもの	AI を 9 回動かすので 1 枚に約 0.2 秒(GPU)。カメラは 1 秒に 30 枚作るが、detector は**1 秒に 1 回、一番新しい画像だけ**を調べる。秒速 7 m で進んでも、カメラは前方約 50 m を写しているので、同じ場所が 5〜7 回は調べられ、見落とさない。計算上は 1 秒に 4 回調べられるが、同じ GPU で Gazebo の描画もしているので余裕を見た

工夫 2:カメラを 30° 前へ傾ける

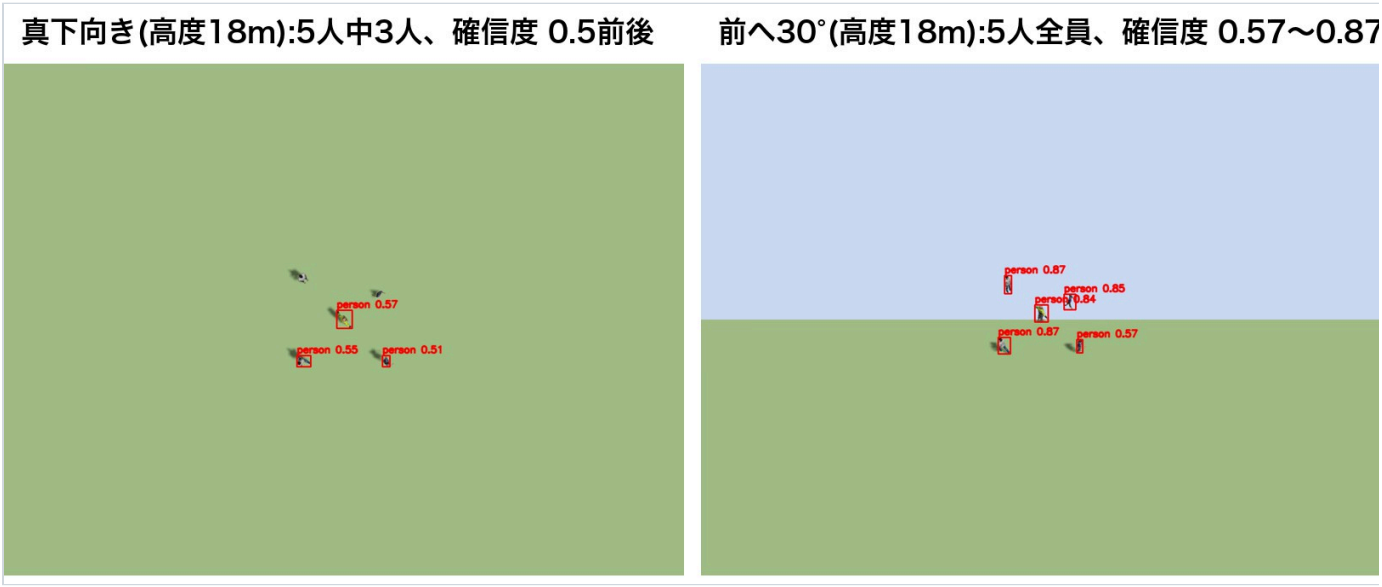


図 3-6 同じ高度 18 m から同じ 5 人を撮った画像(陸上版の実験)。赤い枠が AI の検出結果

困りごと	真下からでは、立っている人や立ち泳ぎの人は頭と肩しか写らず、確信度が 0.5 前後にとどまる
工夫	カメラを真下から前へ 30° 起こした機体を作り、人を斜め上から見る
効果	陸上版:真下では 6 人中 3 人を見逃したが、30° では全員を検出。海難救助版:胸まで水中の人も確信度 0.5 以上で検出
代わりに払うもの	画像の上のほうは遠くを写すので、位置の誤差が大きくなる。そのため 40 m より遠い検出は使わない(工夫 4 の図 3-9)

工夫 3:2 段階確認

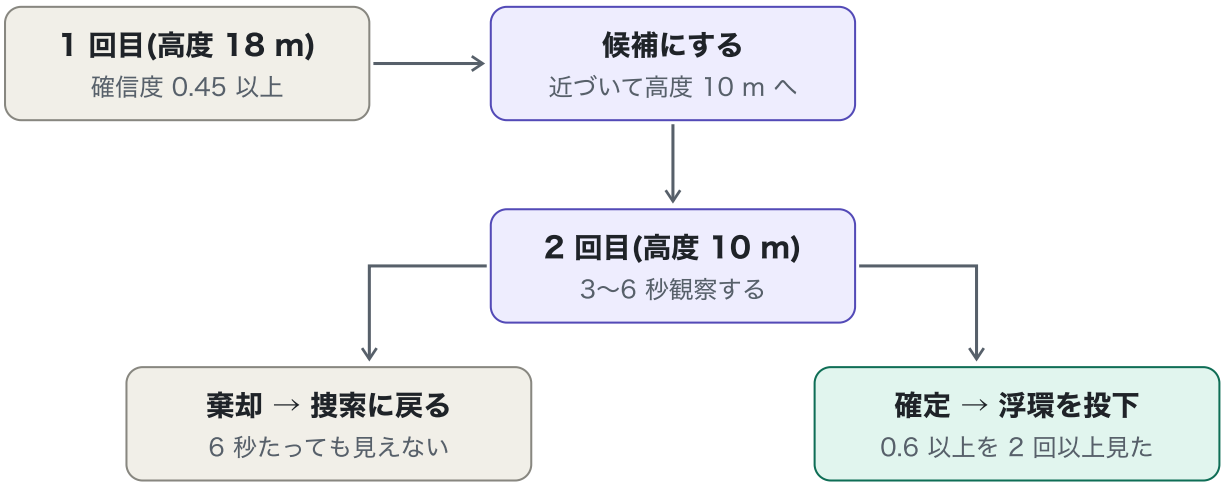


図 3-7 2 段階確認の流れ。遠くからは甘く拾い、近くで厳しく確かめる

困りごと	AI はブイや浮いている木材を人と見間違えることがある。そのまま浮環を落とすと、浮環を無駄にし、本当の人への救助が遅れる
工夫	図 3-7 のように 2 回に分けて判定する
効果	3 回の飛行で誤検出の候補は計 2 件。どちらも 2 回目で棄却し、人ではないものに浮環を落としたことはない
代わりに払うもの	候補ごとに、降りて確かめる時間が 10 秒ほどかかる

工夫 4:機体の傾きを考えて位置を計算する

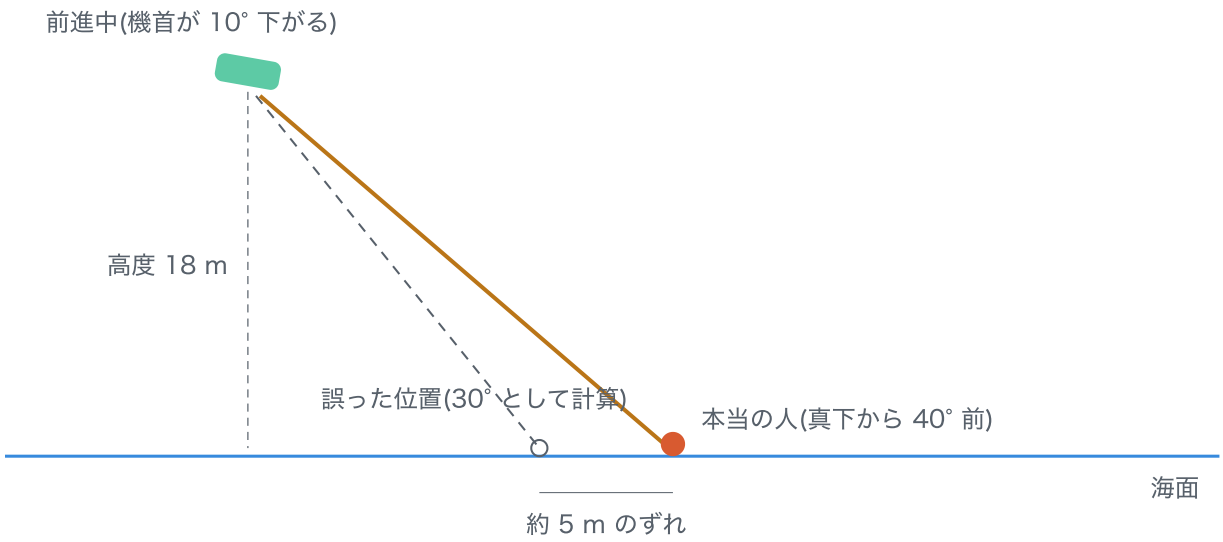


図 3-8 機体の傾きを考えない場合のずれ(18 × tan40° − 18 × tan30° ≈ 4.7 m)

困りごと	前進中、機体は機首を約 10° 下げて傾く。カメラも一緒に傾き、実際には真下から 40° 前を見ている。いつもの 30° として計算すると約 5 m ずれる
工夫	detector は PX4 から機体の傾き(前後・左右・向き)を受け取り、カメラが実際に向いていた方向で計算する。AI の処理に約 0.2 秒かかる間にも機体は動くので、**画像を受け取った瞬間**の位置と傾きを保存しておき、その値で計算する
効果	確定位置の誤差は平均 0.45〜0.65 m

2 次元の画像から、どうして位置(距離)がわかるのか

画像だけでは距離はわかりません。距離を決められるのは、**人は平らな海面(高さ 0 m)にいる、ドローンの高度と傾きはわかっている**、という 2 つの前提があるからです。画像の 1 つの画素は「ある方向」に対応し、その方向に線を伸ばすと、海面のどこか 1 点にぶつかります。それが人の位置です。

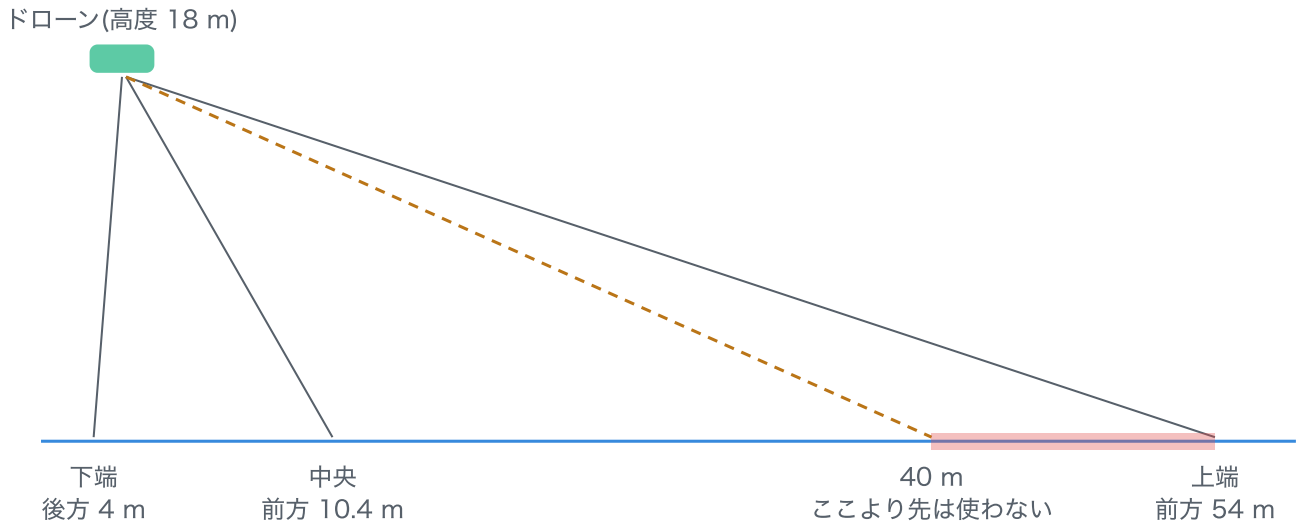


図 3-9 画像の上下の位置と、海面までの距離。上の画素ほど水平に近い方向を向き、遠くで海面にぶつかる

表 3-6 機体の傾きの測定が 1° ずれたときの位置のずれ

写っている場所	ずれ
画像の中央(前方 10 m)	約 0.4 m
画像の上端(前方 54 m)	約 3.1 m

遠くほど、わずかな角度の誤差が大きな距離の誤差になります。そこで、ドローンから 40 m を超える位置は捨てています(detector_node.py の max_range)。なお、この計算は「人が平らな海面にいる」前提に頼るので、船の上に立っている人などは位置がずれます。

3.4 公平な評価:どうやって採点したか

表 3-7 採点に使う 2 つのファイル

ファイル	出力するプログラム	形式	書かれていること
mission_*.json	mission(頭)	JSON(着陸時に 1 回書き出す)	確定した時刻と推定位置、投下した時刻
truth_*.jsonl	truth_logger	JSON Lines(0.5 秒ごとに 1 行追加)	要救助者と浮環の本当の位置

mission(頭)から出力した JSON ファイルの、T1 の部分(3 回目の飛行):

```
{ "id": "T1", "status": "confirmed",
  "first_north": -147.29, "first_east": -30.97, "first_t": 1791880489.39, "first_conf": 0.836,
  "est_north": -148.0,  "est_east": -32.13,  "est_t": 1791880498.60,
  "confirm_t": 1791880498.60, "n_confirm": 3, "max_conf": 0.829,
  "drop": {"t": 1791880503.40, "north": -148.23, "east": -32.6, "alt": 6.08, "ring": "lifering_1"} }
```

項目	意味
first_*	最初に候補として見つけたとき(高度 18 m)の位置・時刻・確信度
est_*	確定したときの推定位置と時刻(高度 10 m の観測を平均)。**採点で比べるのはこの値**
n_confirm、max_conf	確認中に見えた回数(3 回)と最大の確信度
drop	投下した時刻・機体の位置・高度・浮環の名前

truth_logger のファイル(1 行に 1 つの JSON。t が UNIX 時刻、poses が [北, 東]):

```
{"t": 1791880461.618, "poses": {"person_P1": [-152.913, -47.255], "person_P2": [-166.913, -33.254], ...}}
{"t": 1791880462.119, "poses": {"person_P1": [-152.950, -47.319], ...}}
```

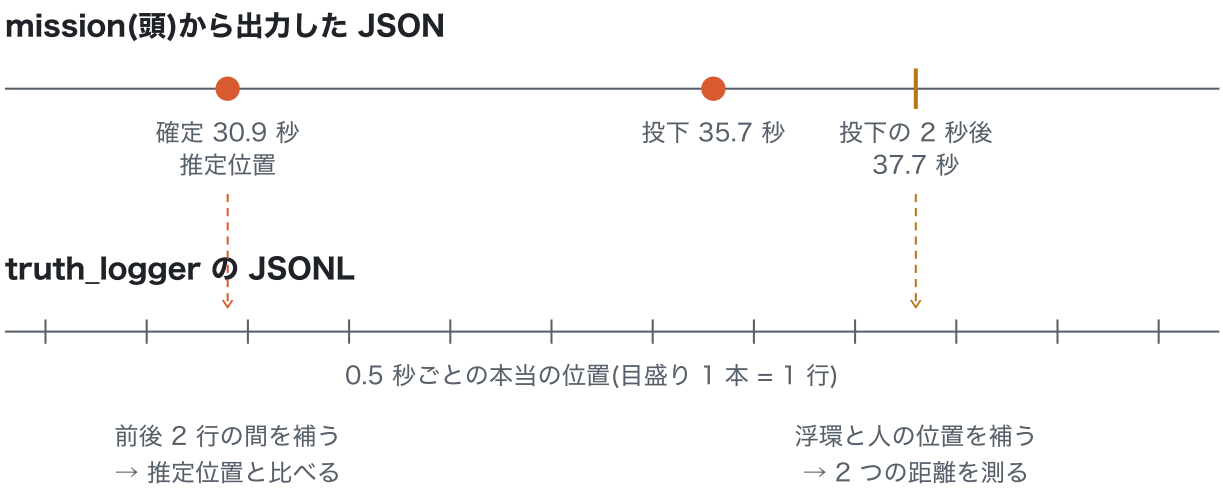


図 3-10 2 つのファイルを時刻で対応させる(図の時刻は出動からの秒数。ファイルの中は UNIX 時刻)

採点の手順(tests/evaluate_mission.py):

1. JSON ファイルから、T1 の確定時刻(30.9 秒)と推定位置を読む
2. truth_logger の記録には 30.9 秒ちょうどの行がないので、**前後 2 行の間を補って**(補間して)本当の位置を求める。例:10.0 秒に北 −152.55 m、10.5 秒に北 −152.59 m なら、10.3 秒は 60% の地点なので −152.55 + (−0.04) × 0.6 = −152.574 m
3. その時刻に T1 の一番近くにいた要救助者(P3)が 5 m 以内なら「T1 は P3 を見つけた」とする。2 つの位置の距離(0.32 m)が**位置の誤差**
4. 投下の 2 秒後(37.7 秒。浮環は約 1.1 秒で着水するので余裕を見た)について、浮環と P3 の本当の位置を同様に求め、その距離(0.21 m)を**浮環と人の距離**とする

どの要救助者とも 5 m 以内で対応しない確定は「誤った確定」として数えます。T 番号は mission が見つけた順に付けた候補番号(棄却した候補にも付く)、P 番号は要救助者の本当の名前で、この対応づけは飛行ごとに変わりま

す(3 回目は T1 = P3)。

4 章 結果

同じ条件で飛ばしても、プログラムどうしのタイミングが毎回わずかにずれるので、結果は少しずつ変わります。たまたまうまくいった 1 回ではないことを示すため、3 回の結果を並べています。

4.1 表の各行の意味

行	意味	どう計算したか
発見した要救助者	5 人中何人を見つけたか	3.4 節の採点で、要救助者と 5 m 以内で対応した確定の数
誤った確定 / 誤検出として棄却	人ではないのに確定した数 / 確認で捨てた数	前者はどの要救助者とも対応しなかった確定、後者は JSON で棄却になった候補
確定位置の誤差 平均 / 最大	推定位置が本当の位置からどれだけずれたか	5 人それぞれの誤差の平均と最大
浮環と人の距離 平均 / 最大	浮環がどれだけ人の近くに落ちたか	投下の 2 秒後の距離。5 個の平均と最大
最初の 1 名 / 全員を確定するまで	出動から何秒で確定したか	確定時刻 − 出動時刻
飛行時間 / 飛行距離	出動から着陸まで	JSON の値

表 4-1 3 回の結果(説明資料の表 1 と同じ)

	1 回目	2 回目	3 回目
発見した要救助者	5 / 5	5 / 5	5 / 5
誤った確定 / 棄却	0 / 0	0 / 1	0 / 1
確定位置の誤差 平均 / 最大	0.45 / 0.77 m	0.49 / 0.92 m	0.65 / 0.99 m
浮環と人の距離 平均 / 最大	0.51 / 0.87 m	0.45 / 1.07 m	0.65 / 1.19 m
最初の 1 名 / 全員を確定	31 / 161 s	31 / 174 s	31 / 189 s
飛行時間 / 距離	216 s / 1021 m	229 s / 1024 m	240 s / 1094 m

4.2 一人ずつの結果(3 回目)

見つけた順	候補番号	要救助者	確定(出動から)	位置の誤差	浮環と人の距離
1	T1	P3	31 秒	0.32 m	0.21 m
2	T2	P2	62 秒	0.32 m	0.24 m
3	T4	P1	85 秒	0.86 m	0.79 m
4	T5	P4	153 秒	0.99 m	1.19 m
5	T6	P5	189 秒	0.74 m	0.81 m
平均				**0.65 m**	**0.65 m**

T3 がないのは、T3 が誤検出の候補で、確認の段階で棄却されたからです。

- **最初の 1 名は 3 回とも 31 秒**:一番近い P3 は捜索基点のそばにいて、そこまでの飛び方は毎回同じ
- **全員の確定は 161〜189 秒とばらつく**:2 人目以降は、それまでの確認・投下の時間や、誤検出の候補への寄り道の有無で前後する(3 回目は T3 への寄り道で 1 回目より 28 秒遅い)
- **浮環は 15 個すべて 1.2 m 以内**:手を伸ばして届く目安を 2 m とすると、すべて届く範囲

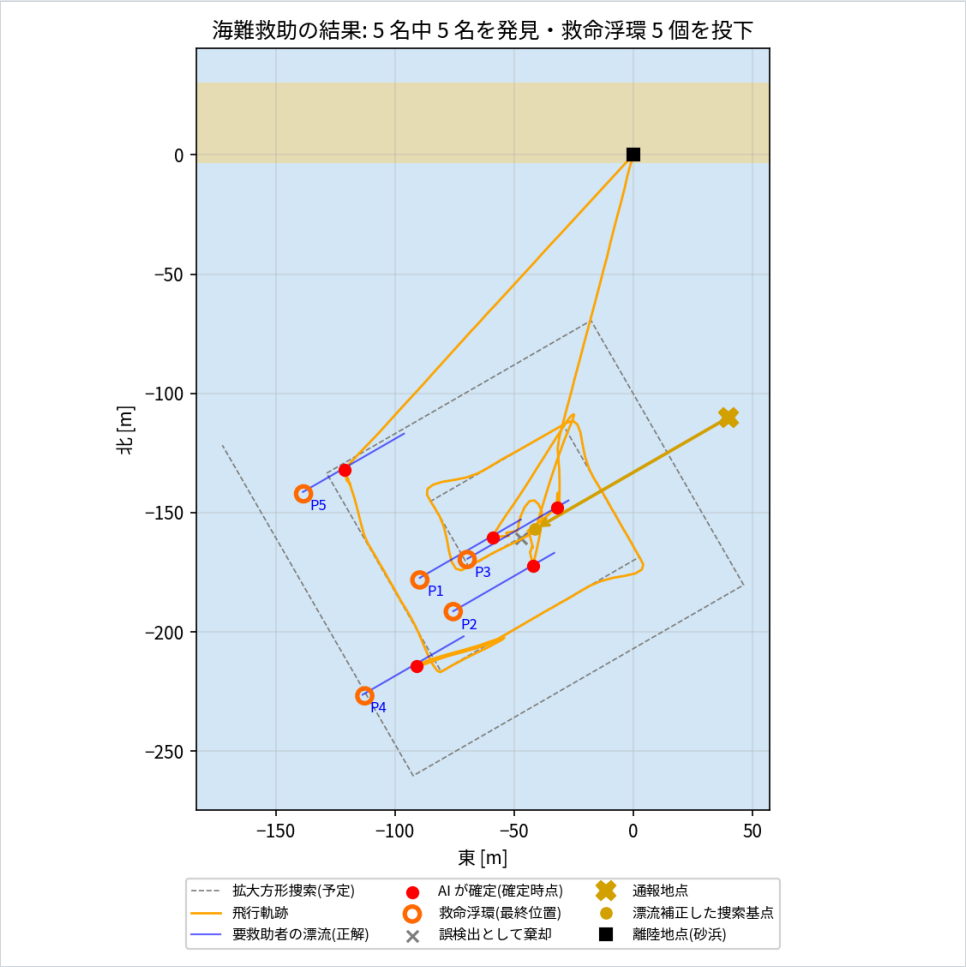


図 4-1 3 回目の結果。青線は要救助者の漂流(本当の位置、5 本とも平行で同じ長さ=同じ海流)、赤点は確定した時点の位置(青線の途中に乗る)、オレンジの輪は浮環(人と一緒に流れる)、点線の四角は拡大方形捜索の予定経路

> 説明資料 4 章の「漂流補正がなければ、最初の数分は空振りになる」は、**実験で確かめた結果ではなく推論**です。根拠は、補正しない場合の捜索の中心が人の集団から約 107 m 離れていたこと。厳密には、補正なしの飛行を行って比べる必要があります(5 章)。

4.3 誤差はどこから生まれるのか

シミュレーションには波も風もなく、海流も設定どおりです。それでも誤差が出るのは、ドローンの「測り方」そのものにずれの原因があるからです。

#	原因	内容
①	AI の枠の中心は「海面の点」ではない	枠が囲むのは水面から出ている頭や胸で、中心は海面より 0.5 m ほど上。斜めから見ているので、その方向に線を伸ばすと海面にぶつかる点は人の少し奥になる(視線 40° で約 0.4 m)
②	PX4 が知っている機体の位置も推定値	PX4 は模擬 GPS や加速度センサーから機体の位置を推定しており、本当の位置と完全には一致しない。人の位置 = 機体の位置 + 方向と距離なので、そのまま人の位置のずれになる
③	画像の時刻と、機体の位置の時刻の食い違い	画像が detector に届くまでのわずかな遅れの間に、機体の位置や傾きが変わる
④	AI の枠そのもののずれ	枠は人の輪郭と 1〜2 画素ずれる。高度 10 m では 1 画素が数 cm なので影響は小さめ

浮環と人の距離は、「mission が思っている人の位置(①〜④の誤差を含む)」と「その真上でドローンが止まった位置(目的地から 1.0 m 以内なら到着とみなす。位置は②の推定値)」の組み合わせで決まります。風がないので浮環はまっすぐ落ち、着水後は人と同じ海流で流れるので、着水した瞬間の距離がそのまま保たれます。

4.4 後から見つけた人ほど誤差が大きい理由(未確定)

表 4-2 3 回分の一人ずつの位置の誤差(かっこ内は見つけた順)

要救助者	姿勢	1 回目	2 回目	3 回目
P3	立ち泳ぎ	0.18 m(1)	0.15 m(1)	0.32 m(1)
P2	立ち泳ぎ	0.40 m(2)	0.32 m(3)	0.32 m(2)
P1	浮いている	0.37 m(3)	0.54 m(2)	0.86 m(3)
P5	浮いている	0.53 m(4)	0.51 m(4)	0.74 m(5)
P4	立ち泳ぎ	0.77 m(5)	0.92 m(5)	0.99 m(4)

- ①が主な原因なら、体が高く出ている**立ち泳ぎの人ほど誤差が大きい**はず。しかし立ち泳ぎの P3・P2 は最も小さく、P4 は最も大きい。姿勢だけでは説明できない
- **見つけた順番**で見ると 3 回ともそろっている(1 番目の P3 が最小、4〜5 番目の P4・P5 が最大)
- 判断:①はどの人にも 0.2〜0.4 m 程度のずれを与えている可能性が高い。順番の傾向は、時間とともに増える②(PX4 の推定のずれ)のほうが合う。ただし、順番と姿勢・場所が重なっているので、このデータだけでは切り分けられない

表 4-3 原因を確かめる方法(未実施)

確かめたいこと	方法
① 枠の中心の高さによるずれ	誤差の**向き**を調べる。①なら、ずれはいつも「ドローンから見て奥」にそろう
② PX4 の推定のずれ	truth_logger にドローンの本当の位置も記録させ、PX4 の推定位置との差が時間とともに増えるかを見る
順番と姿勢を切り離す	人の配置を入れ替えて(例:P4 を基点の近くに置いて)飛ばし、P4 の誤差が小さくなるかを見る

5 章 考察と今後の課題

5 章は、「今回**単純にしたこと**」と「実際の海で使うには**何が足りないか**」の整理です。

5.1 説明資料に書いた 4 項目

項目	今回単純にしたこと	実際の海では	必要なこと
海流の不確かさ	ドローンが使う予報と、人を流す海流が完全に同じ	予報には誤差があり、場所や時間でも流れが変わる	いそうな範囲を確率で表し、確率の高い所から探す(図 5-1)
見え方の変化	おだやかな海面、昼間の明るい条件だけ	波・白波・照り返し・夜間で見え方が大きく変わり、見間違いも増える	実際の海の画像で AI を追加学習する。夜間は赤外線カメラを組み合わせる
投下の物理	浮環はまっすぐ落ち、着水後は人と同じ海流で流れる	浮環は軽く風を受けやすいので、人より速く流れて離れていく	風を測り、その分だけ風上側に落とす
運用	確定位置を緯度・経度と漂流予測つきで JSON に書き出す	位置をその場で救助側に届ける必要がある	救助船への送信、複数機での分担(フォイルボード版につながる)

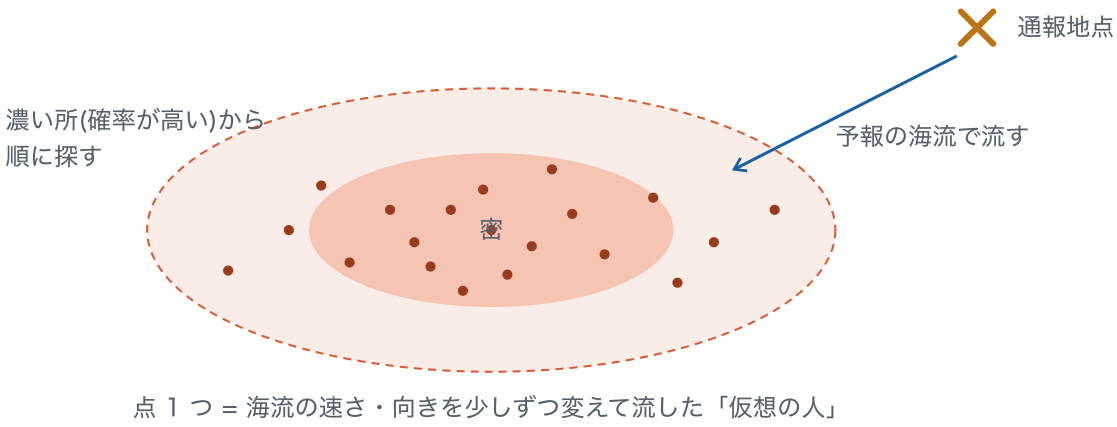


図 5-1 確率で探す考え方(模式図)。海流の速さや向きを少しずつ変えた仮想の人を何千人も流し、集まる所から順に探す。米国沿岸警備隊の捜索計画システム(SAROPS)がこの考え方に基づく

5.2 理解の過程で見つかった課題

課題	内容と対策
散らばりを初期配置で作っている	時間とともに広がる様子は再現していない(1.4 節)。人ごとに漂流の速さ・向きを少し変えれば、予報とのずれも生まれ、漂流予測の誤差に強いかを試せる
固定した捜索経路から人が離れていく	予報の誤差が大きいと見つからない(表 3-2)。捜索経路を海流で流す、脚を増やす、見つけた人の動きから海流を推定し直す
漂流補正の効果を比べていない	補正なしの飛行を行い、最初の発見までの時間などを比べる
誤差の原因を切り分けていない	後から見つけた人ほど誤差が大きい理由(4.4 節)を、表 4-3 の方法で確かめる

6 章 使用した技術

シミュレーションと飛行

名前	役割	図 2-1 では
Gazebo Harmonic 8	海・人・ドローン・カメラ映像を作る仮想の世界	世界
VelocityControl	Gazebo の機能。モデルに一定の速度を与え続ける。要救助者を海流で流すのに使用	世界の中
ShaderParam	Gazebo の機能。海面の波の模様を描く	世界の中
PX4-Autopilot v1.18(SITL)	ドローンの飛行制御ソフト。実機と同じソフトをパソコン上で動かしたもの(SITL = 実機なしで動かすこと)	手足
自作の機体設定 4099	PX4 に登録した、カメラを 30° 傾けた捜索用の機体の設定。地上局なしで飛べる設定も含む	手足

プログラム同士の通信

名前	役割
ROS 2 Humble	複数のプログラムを動かし、データを受け渡しさせる仕組み(実装編で扱う)
Micro XRCE-DDS	PX4 と ROS 2 の間をつなぐ通訳。mission の仮の目標点はこれを通って PX4 に届く
ros_gz_image	Gazebo のカメラ画像を、ROS 2 で受け取れる形に変換して流す
gz-transport(Python)	Python から直接 Gazebo を操作する仕組み。浮環の出現・移動と、本当の位置の記録に使用

AI

名前	役割
YOLO11m / YOLO11s	画像から人を見つける AI。m は大きめで高精度(GPU 用)、s は小さめで速い(GPU がないときの予備)
COCO 学習済み	公開の写真集 COCO(人・車・動物など 80 種類)で学習を済ませた状態。追加の学習はしていない
ONNX	AI のモデルを保存する共通の形式。特定の道具がなくても動かせるように変換した
ONNX Runtime GPU 1.20	ONNX 形式の AI を GPU で動かす道具

画像・図・素材と環境

名前	役割
OpenCV	画像の切り分け・縮小、検出の枠を描く、表示画面のウィンドウ
Pillow	表示画面やタイトル画面に日本語の文字を書く
matplotlib	結果の地図(図 4-1)を描く
Gazebo Fuel	Gazebo の公開モデル集。人・ボート・桟橋・漁船・ブイ・海面のモデルを借りた
実行環境	Ubuntu 22.04 のノート PC(NVIDIA RTX 3060 Laptop GPU)。シミュレーション・AI・録画はすべてこの PC

(実装編:ROS 2、YOLO、座標の計算、PX4、フォイルボード版の差分 は、このあとに追加予定)